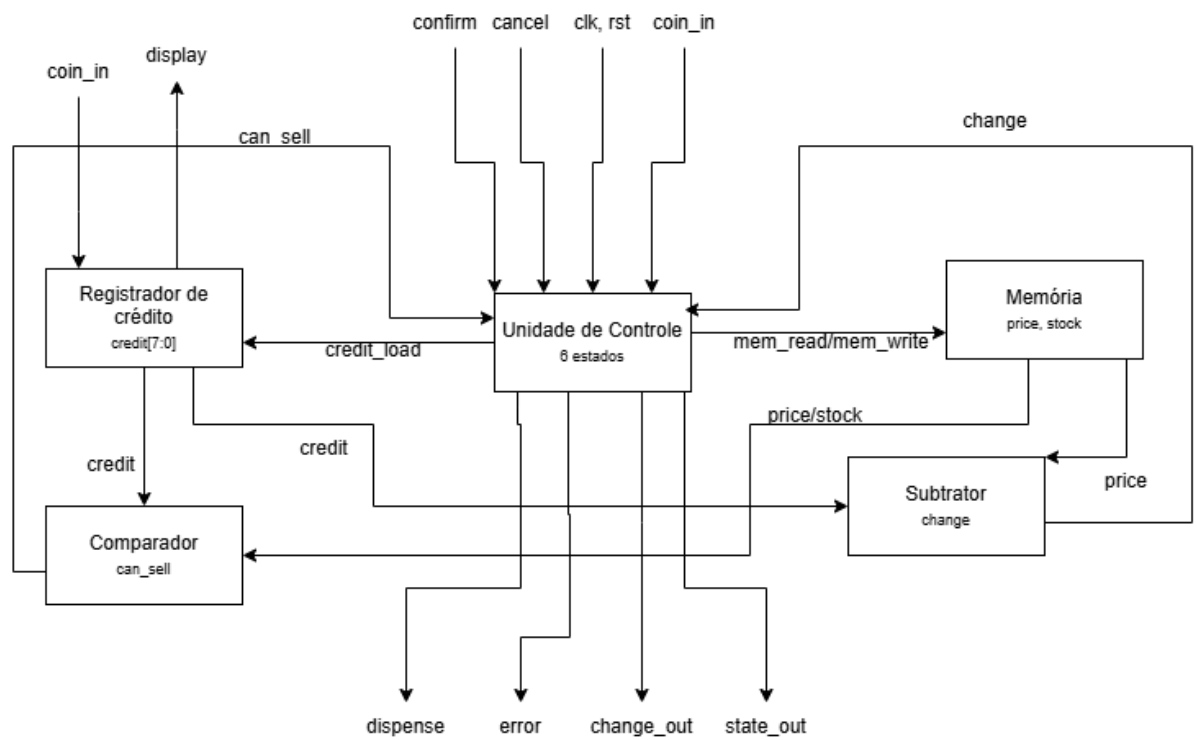


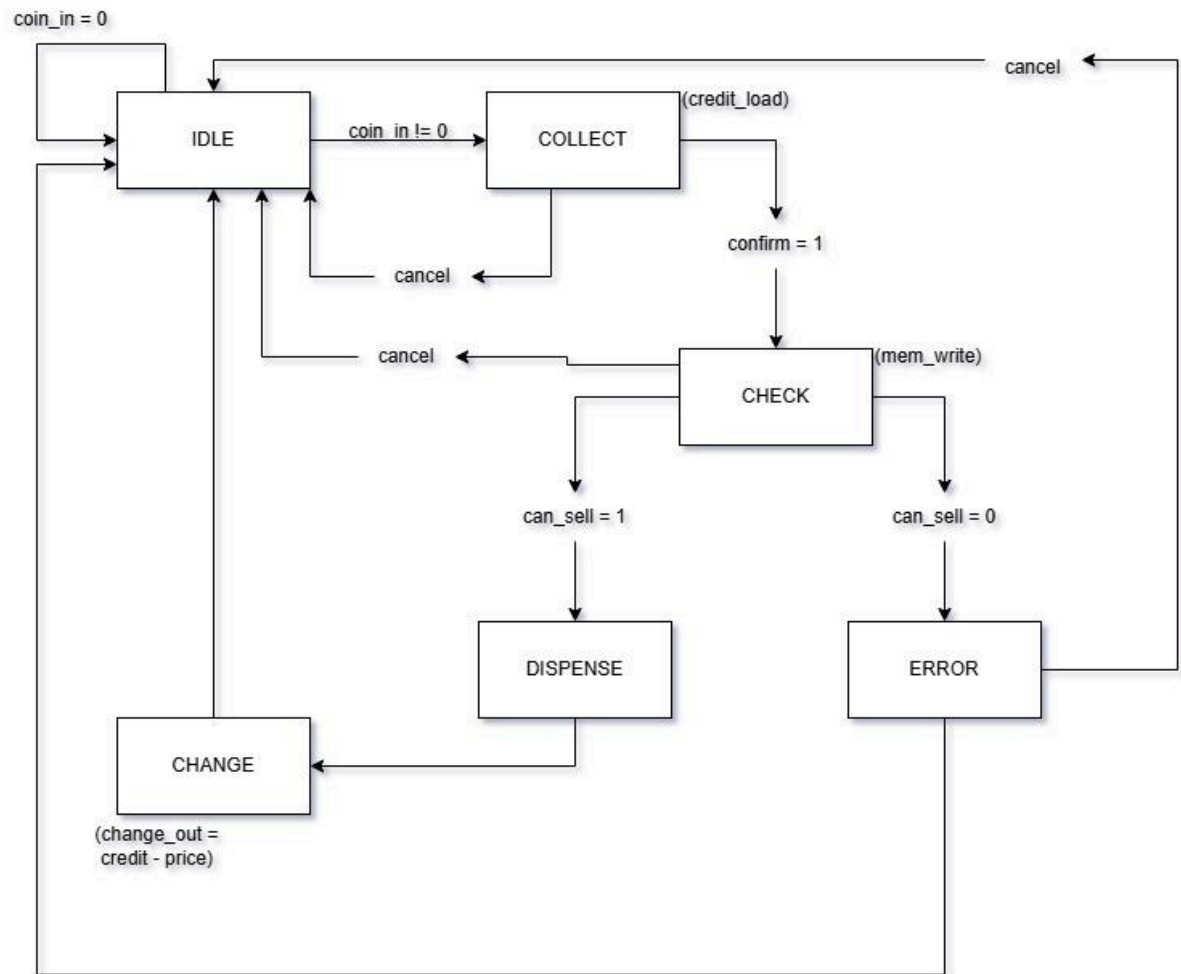
1. Link para o github

https://github.com/emilieny/emilieny_victor_vending

2. Diagrama de blocos



3. Diagrama de Estados



4. Descrição dos módulos

A máquina de vendas foi projetada previamente separando a unidade de controle e o caminho de dados, para garantir modularidade e facilitando o desenvolvimento, separando o desenvolvimento em 7 arquivos principais:

Unidade de controle (control_unit.sv)

Implementação da Máquina de Estados Finitos (FSM), onde foram feitos 2 blocos **always**, um sequencial **always_ff**, responsável pela atualização do estado na borda de subida do clock e um combinacional **always_comb**, responsável por definir o próximo estado e as saídas de controle.

Separando a lógica sequencial da combinacional facilita a leitura e evita interferência, além de que dividindo os estados passo a passo, evita condições de corrida e garante que a FSM funcione de forma síncrona.

Acumulador de Crédito (credit_reg.sv)

Responsável por receber as moedas inseridas, convertê-las para seu valor real em centavos e manter o saldo atual do usuário.

Foi criada uma função **coin_value** para converter a entrada **coin_in** para valores de 8 bits correspondentes aos centavos. A soma do crédito ocorre de forma síncrona quando o sinal **credit_load** é habilitado em IDLE ou COLLECT.

Isolando o registro e a soma do crédito fora do FSM simplifica a máquina de estados, o **credit_load** atua como um enable, garantido que adição de moedas fora do estado do collect não corrompa o cálculo do troco ou da venda.

Memória de Produtos (memory.sv)

Responsável por armazenar os preços dos produtos e a quantidade do estoque de cada um.

A memória foi feita como um array de 4 posições, onde cada uma possui 16 bits, 8 bits guarda o preço e os outros 8 guardam o estoque. A leitura de price e stock foi feita de forma combinacional em **always_comb** enquanto a escrita foi feita de forma síncrona com **always_ff**

A leitura combinacional garante que os valores de preço e estoque estejam sempre disponíveis para o comparador e para o subtrator no mesmo ciclo de clock, evitando atrasos na FSM, a escrita síncrona ocorre apenas quando **mem_write** é ativado e garante que o estoque diminua de forma controlada após a compra ser validada.

Comparador (comparator.sv)

Módulo para verificar a validade da transação, o módulo incrementa apenas a lógica combinacional para dar o feedback a FSM durante a fase de CHECK, para verificar se é possível comprar o produto devido ao valor do preço e sua disponibilidade no estoque, assim definindo se a FSM passa do estado de CHECK para DISPENSE ou ERROR.

Subtrator / Calculo de Troco (subtractor.sv)

Calcula o valor do troco a ser devolvido. Implementado de forma combinacional, o circuito realiza o cálculo de **credit - price** para uma compra normal ou devolve o valor de **credit** caso tenha um pedido de cancelamento, chamado por **cancel_req**. Incluímos uma proteção que retorna zero se o crédito for menor que o preço.

O cálculo combinacional garante que o troco seja feito antes da FSM verificar sua validação, impedindo que o sistema emita algum valor inesperado para o troco do usuário.

Módulo Top-level (vending_top.sv)

Módulo que unifica os outros módulos, instanciando as variáveis e criando roteamento dos fios, nele também foi adicionado um registrador para o troco **change_out**, que só é atualizado quando sinaliza que o troco é válido com **w_change_valid**.

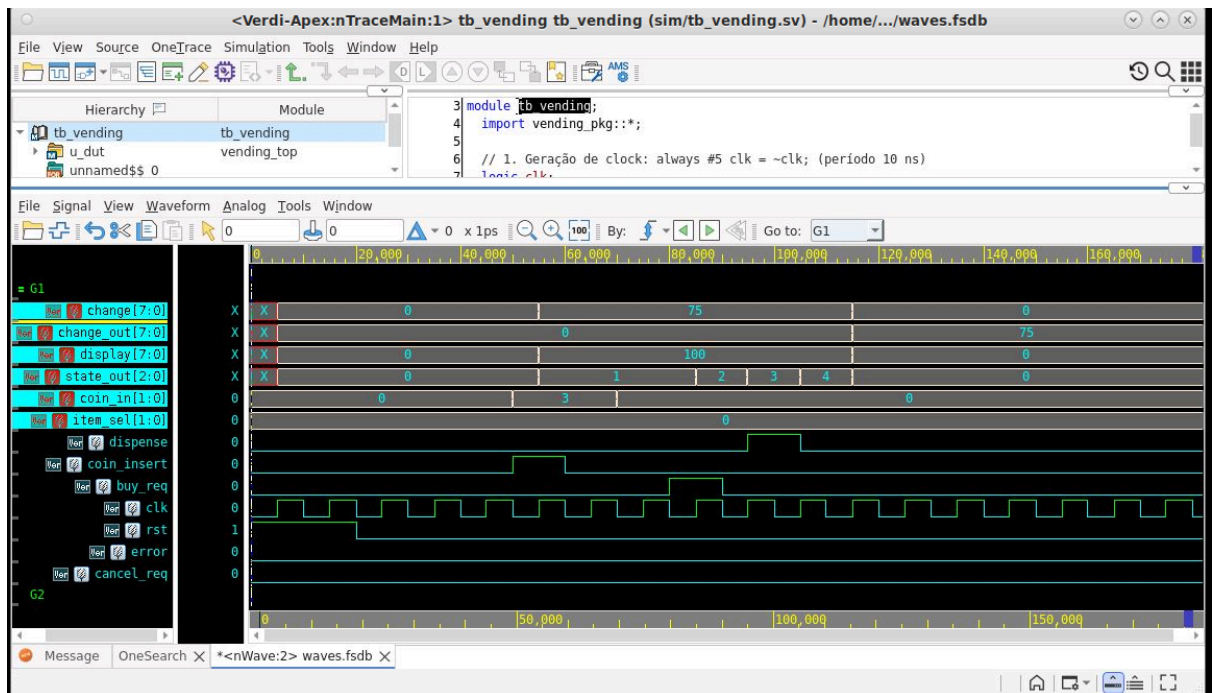
Registrando a saída evita que ruídos ou flutuações lógicas, fazendo com que o usuário só receba o valor do troco quando finaliza a compra ou quando ela é cancelada.

Pacote (Vending_pkg.sv)

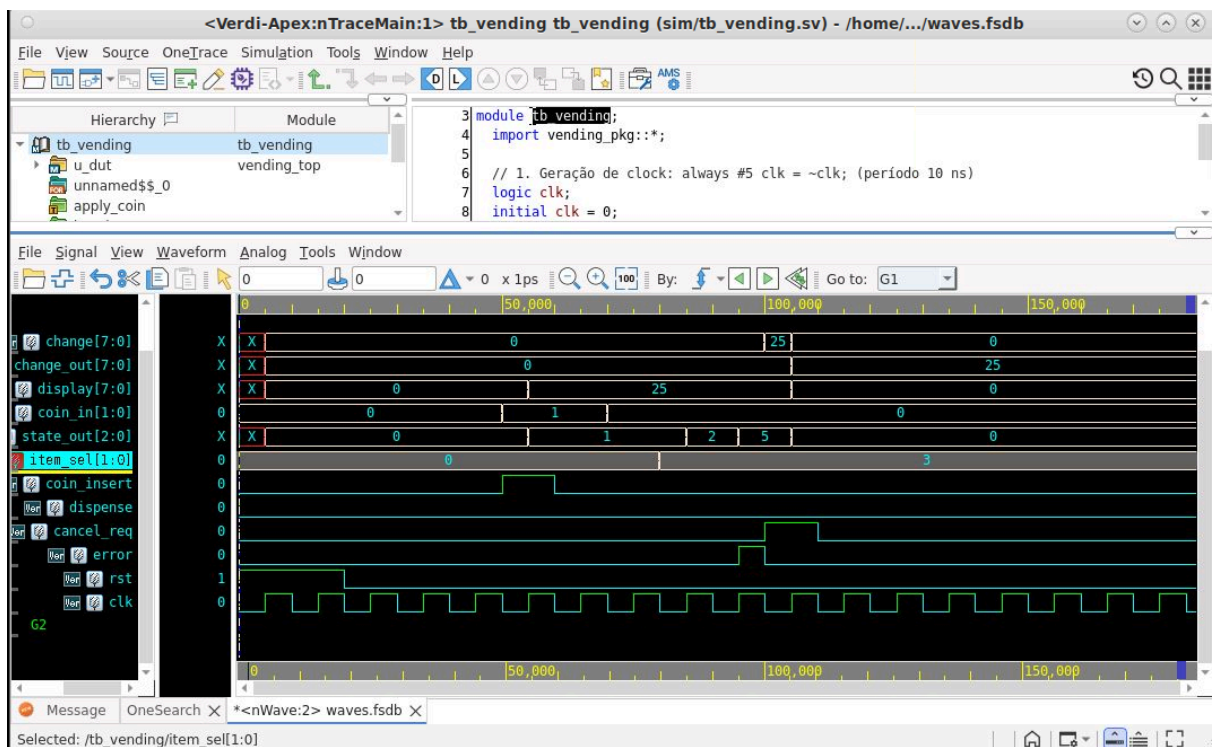
apenas um pacote para definir os estados da FSM em um só lugar, evitando de precisar repetir o mesmo bloco nos outros módulos múltiplas vezes, evitando redundâncias e inconsistências.

5. Waveforms

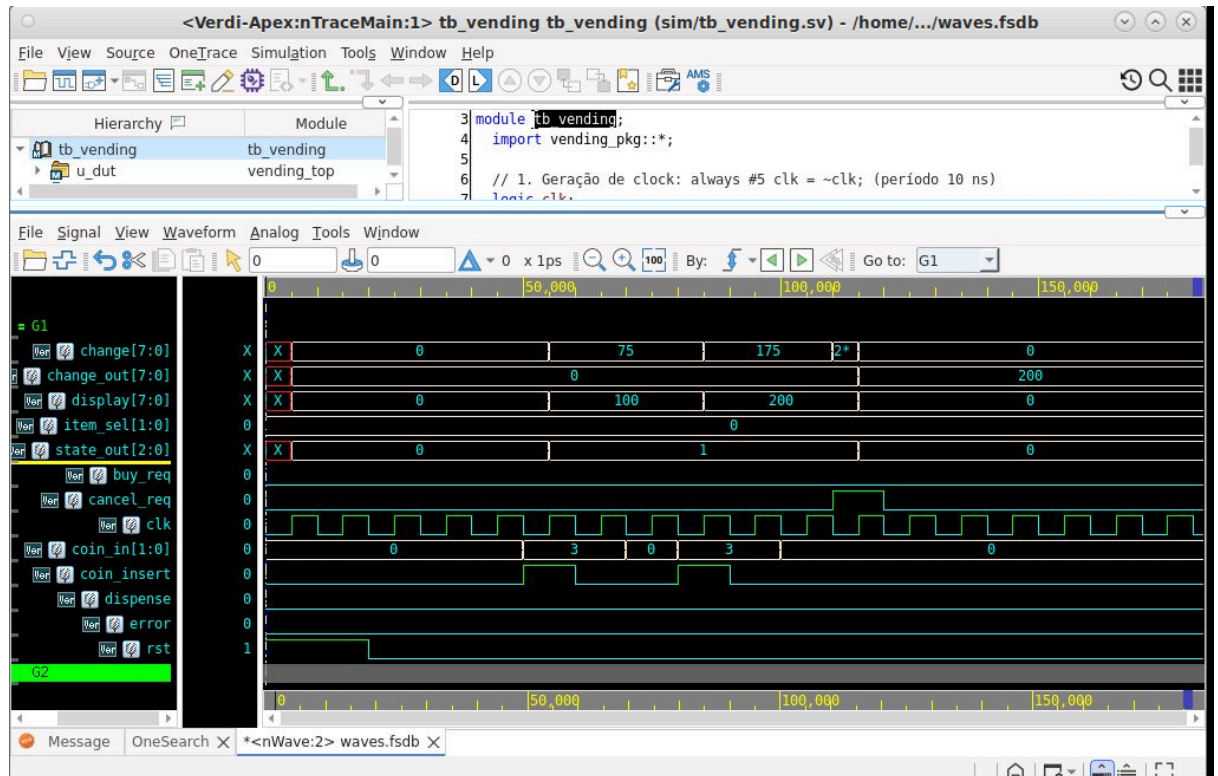
Cenário 1



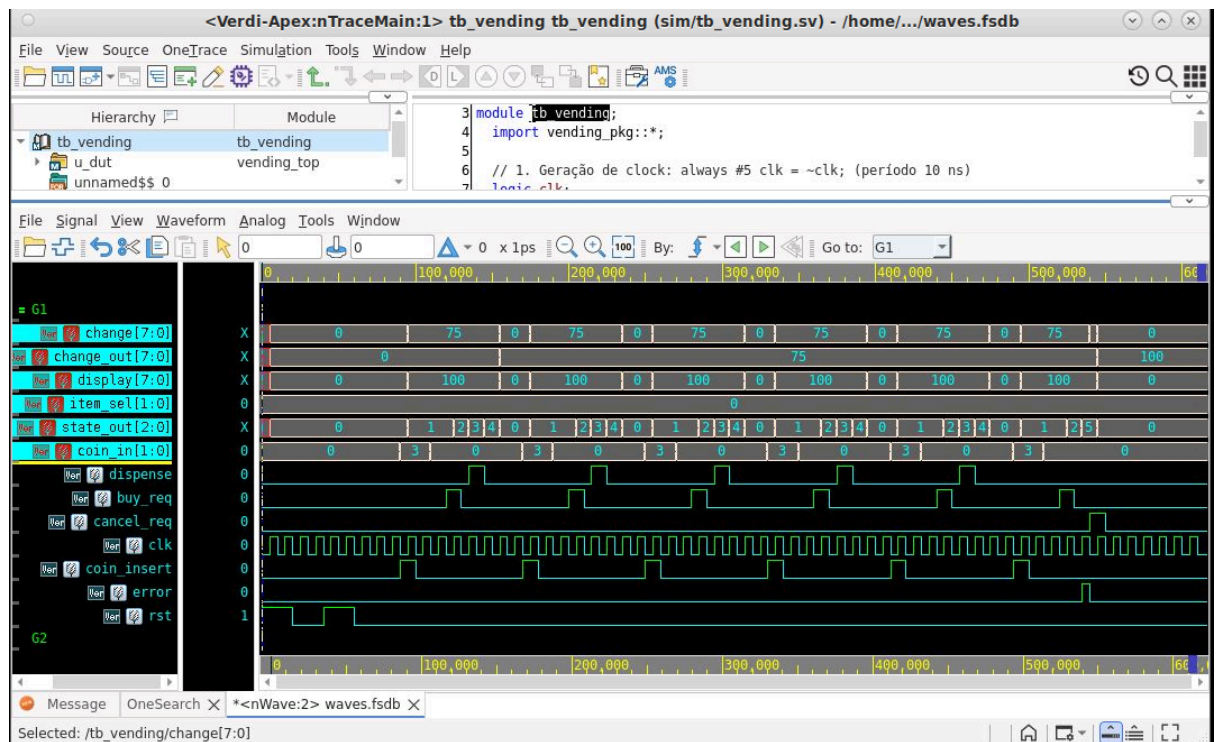
Cenário 2



Cenário 3



Cenário 4



6. Saída do testbench

```
Terminal
Arquivo Editar Exibir Pesquisar Terminal Ajuda
Verdi KDB elaboration done and the database successfully generated
./simv
Info: [VCS SAVE RESTORE INFO] ASLR (Address Space Layout Randomization) is detected on the machine. To enable $save functionality, ASLR will be switched off and simv re-executed.
Please use '-no_save' simv switch to avoid re-execution or '-suppress=ASLR_DETECTED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP2_Full64; Runtime version X-2025.06-SP2_Full64; Jul 10 13:55 2026
*Verdi* Loading libsscore_vcs202506.so
FSDB Dumper for VCS, Release Verdi_X-2025.06-SP2, Linux x86_64/64bit, 11/27/2025
(C) 1996 - 2025 by Synopsys, Inc.
*Verdi* : Create FSDB file 'waves.fsdb'
*Verdi* : Begin traversing the scope (tb_vending), layer (0).
*Verdi* : End of traversing.

=====
INICIANDO TESTES VENDING
=====

-> Cenario 1: Compra bem-sucedida (Cafe com R$ 1,00)
[PASS] Cenario 1: dispense=1 (DISPENSE)
[PASS] Cenario 1: change_out=75 (CHANGE)
[PASS] Cenario 1: credit=0 ao final
[PASS] Cenario 1: FSM retornou a IDLE

=====
TESTES CONCLUIDOS
=====

$finish called from file "sim/tb_vending.sv", line 221.
$finish at simulation time 180000
V C S   S i m u l a t i o n   R e p o r t
Time: 180000 ps
CPU Time: 0.430 seconds; Data structure size: 0.0Mb
Fri Jul 10 13:55:31 2026
[emilienv.silva@srv-microelettronica3 emilienv victor vending]$
```

```
Terminal
Arquivo Editar Exibir Pesquisar Terminal Ajuda
CPU time: 1.095 seconds to compile + .557 seconds to elab + .593 seconds to link
Verdi KDB elaboration done and the database successfully generated
./simv
Info: [VCS SAVE RESTORE INFO] ASLR (Address Space Layout Randomization) is detected on the machine. To enable $save functionality, ASLR will be switched off and simv re-executed.
Please use '-no_save' simv switch to avoid re-execution or '-suppress=ASLR_DETECTED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP2_Full64; Runtime version X-2025.06-SP2_Full64; Jul 10 13:59 2026
*Verdi* Loading libsscore_vcs202506.so
FSDB Dumper for VCS, Release Verdi_X-2025.06-SP2, Linux x86_64/64bit, 11/27/2025
(C) 1996 - 2025 by Synopsys, Inc.
*Verdi* : Create FSDB file 'waves.fsdb'
*Verdi* : Begin traversing the scope (tb_vending), layer (0).
*Verdi* : End of traversing.

=====
INICIANDO TESTES VENDING
=====

-> Cenario 2: Credito insuficiente (Snack com R$ 0,25)
[PASS] Cenario 2: error=1 ativado
[PASS] Cenario 2: FSM foi para ERROR (3'b101)

=====
TESTES CONCLUIDOS
=====

$finish called from file "sim/tb_vending.sv", line 221.
$finish at simulation time 180000
V C S   S i m u l a t i o n   R e p o r t
Time: 180000 ps
CPU Time: 0.450 seconds; Data structure size: 0.0Mb
Fri Jul 10 13:59:25 2026
[emilienv.silva@srv-microelettronica3 emilienv victor vending]$
```



```
Terminal
Arquivo Editar Exibir Pesquisar Terminal Ajuda

./simv
Info: [VCS_SAVE_RESTORE_INFO] ASLR (Address Space Layout Randomization) is detected on the machine. To enable $save functionality, ASLR will be switched off and simv re-executed.
Please use '-no_save' simv switch to avoid re-execution or '-suppress=ASLR_DETECTED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP2_Full64; Runtime version X-2025.06-SP2_Full64; Jul 10 14:00 2026
*Verdi* Loading libsscore vcs202506.so
FSDB Dumper for VCS, Release Verdi_X-2025.06-SP2, Linux x86_64/64bit, 11/27/2025
(C) 1996 - 2025 by Synopsys, Inc.
*Verdi* : Create FSDB file 'waves.fsdb'
*Verdi* : Begin traversing the scope (tb_vending), layer (0).
*Verdi* : End of traversing.

=====
INIICIANDO TESTES VENDING
=====

-> Cenario 3: Cancelamento apos inserir R$ 2,00
[PASS] Cenario 3: Credito acumulado e de 200 centavos
[PASS] Cenario 3: change_out=200 no cancelamento
[PASS] Cenario 3: credit=0 apos cancel
[PASS] Cenario 3: FSM retornou a IDLE (3'b000)

=====
TESTES CONCLUIDOS
=====

$finish called from file "sim/tb_vending.sv", line 221.
$finish at simulation time 180000
V C S S i m u l a t i o n R e p o r t
Time: 180000 ps
CPU Time: 0.440 seconds; Data structure size: 0.0Mb
Fri Jul 10 14:00:37 2026
[emilienv.silva@srv-microeletronica3 emilienv victor vending]$
```

```
Terminal
Arquivo Editar Exibir Pesquisar Terminal Ajuda

*Verdi* Loading libsscore vcs202506.so
FSDB Dumper for VCS, Release Verdi_X-2025.06-SP2, Linux x86_64/64bit, 11/27/2025
(C) 1996 - 2025 by Synopsys, Inc.
*Verdi* : Create FSDB file 'waves.fsdb'
*Verdi* : Begin traversing the scope (tb_vending), layer (0).
*Verdi* : End of traversing.

=====
INIICIANDO TESTES VENDING
=====

-> Cenario 4: Estoque zerado (Comprar cafe 5 vezes e falhar na 6a)
Comprando cafe 1/5 (estoque restando)...
Comprando cafe 2/5 (estoque restando)...
Comprando cafe 3/5 (estoque restando)...
Comprando cafe 4/5 (estoque restando)...
Comprando cafe 5/5 (estoque restando)...
Tentando a 6a compra de cafe (deve falhar por falta de estoque)...
[PASS] Cenario 4: error=1 (stock=0 na 6a tentativa)
[PASS] Cenario 4: FSM foi para ERROR (3'b101)
[PASS] Cenario 4: change_out=100 apos cancelar do ERROR
[PASS] Cenario 4: credit=0 apos cancel
[PASS] Cenario 4: FSM retornou a IDLE (3'b000)

=====
TESTES CONCLUIDOS
=====

$finish called from file "sim/tb_vending.sv", line 221.
$finish at simulation time 610000
V C S S i m u l a t i o n R e p o r t
Time: 610000 ps
CPU Time: 0.460 seconds; Data structure size: 0.0Mb
Fri Jul 10 14:01:48 2026
[emilienv.silva@srv-microeletronica3 emilienv victor vending]$
```

7. Resultado de síntese

Usando `compile_ultra -no_autoungroup` :

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 Mhz	15.53	799.447410	0.97 (MET)
18 ns	55.5 Mhz	13.53	799.447410	0.97(MET)
16 ns	62.5 Mhz	11.53	799.447410	0.97 (MET)
14 ns	71.4 Mhz	9.53	799.447410	0.97 (MET)
12 ns	83.3 Mhz	7.53	799.447410	0.97 (MET)
10 ns	100 Mhz	5.53	799.447410	0.97 (MET)
8 ns	125 Mhz	3.53	799.447410	0.97 (MET)
6 ns	166.7 Mhz	1.53	799.447410	0.97 (MET)
4 ns	250 Mhz	- 0.01	1005.020787	0.51 (VIOLATED)
2 ns	500 Mhz	- 2.47	799.447410	0.97 (VIOLATED)

- Período mínimo suportado: 6 ns
- Período abaixo do mínimo: 4ns, viola o timing e aumenta a área, mas na terceira mudança ele consegue gerar um slack (ns) = 0.00 (MET) graças ao aumento da área
- Ao retirar o `-no-autoungroup` da síntese e usando o período mínimo suportado (6 ns), há uma mudança de 1.53 ns para 1.70 no slack, 0.97 para 0.80 no timing e 799.447410 para 759.399597 na área.

Usando `compile_ultra -area_effort high` :

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 Mhz	16.41	330.489588	0.09 (MET)
18 ns	55.5 Mhz	14.41	330.489588	0.09 (MET)
16 ns	62.5 Mhz	12.41	330.489588	0.09 (MET)
14 ns	71.4 Mhz	10.41	330.489588	0.09 (MET)
12 ns	83.3 Mhz	8.41	330.489588	0.09 (MET)
10 ns	100 Mhz	6.41	330.489588	0.09 (MET)

Período	Frequência	Slack(ns)	Área	Timing
8 ns	125 Mhz	4.41	330.489588	0.09 (MET)
6 ns	166.7 Mhz	2.41	330.489588	0.09 (MET)
4 ns	250 Mhz	0.41	330.489588	0.09 (MET)
2 ns	500 Mhz	- 1.59	330.489588	0.09 (VIOLATED)

8. Análise

1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

O caminho crítico é o caminho de decisão da FSM em função do datapath.

Módulos que o caminho crítico atravessa:

- registrador de crédito em credit_reg.sv;
- comparador em comparator.sv;
- lógica combinacional da FSM em control_unit.sv;
- e o registrador de estado da FSM..

2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

À medida que o período de clock é reduzido, a área total tende a aumentar. Isso acontece porque o sintetizador passa a usar células mais rápidas, inserir buffers e, em alguns casos, aumentar o paralelismo interno da lógica para reduzir o atraso e fechar timing.

3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?

O Design Compiler não conseguiu mais fechar o timing na faixa de aproximadamente 166 MHz, o que corresponde a um período de cerca de 6 ns. A partir desse ponto, o slack deixa de ser positivo e passa a ficar negativo, indicando que o caminho crítico já não consegue completar a propagação em tempo dentro do período imposto.

4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?

O estado CHECK, principalmente na transição de DISPENSE ou ERROR. Pois, nesse estado o circuito precisa usar o crédito acumulado, ler o preço e estoque da memória, executar a comparação e esperar a FSM decidir o próximo estado.

5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

A inserção de um registrador intermediário para armazenar a decisão de can_sell e fazer a FSM usar esse resultado no ciclo seguinte, para não depender diretamente do resultado combinacional no mesmo ciclo.

9. conclusão

Nossa maior dificuldade inicial foi analisar as ondas para identificar os erros existentes pois estávamos tendo problemas com o troco e com o fluxo da máquina de estados, após conseguir entender bem, conseguimos arrumar os problemas e ajustar o fluxo do funcionamento. Além de saber como definir os clocks durante a fase de testbench